SOEN 343 - Phase 3
GOF design pattern and class diagram

SUMMS - Smart Urban Mobility Management System

Team name: TABAC

Lead: Daniel Ganchev - 40315755
Developer 1: Adam Ousmer - 40246695
Architect 1: Dylan Bourret - 40287207
Developer 2: Yanis Djeridi - 40227313
Architect 2: Emre Emuler - 40212481
Tester: Abed-Elmouhsen Cherkawi - 40323359

March 26th 2026

**Introduction**.....3
**1. Project Overview**.....3
1.1 Brief description of the system-to-be.....3
1.2 Architectural style selected.....3
1.3 Core components and responsibilities.....4
1.3.1 Components.....4
1.3.2 Responsibilities.....4
**2. Design Patterns**.....6
**3 Updated Class Diagram**.....9

# Introduction

This assignment targets:

1. Understanding the importance of design pattern.
2. Refactoring the class diagram.
3. Continuing the implementation

# 1. Project Overview

## 1.1 Brief description of the system-to-be

The main goal of the Smart Urban Mobility Management System (SUMMS) is to unite public and private transportation services into one interface for commuters. The system will allow:

- Commuters to plan multimodal trips according to their needs:
    - Vehicle rentals (cars/EVs, bikes/ebikes, and scooters)
    - External Services:
        - Public Transit
        - Parking Service
- City admins to monitor traffic
    - External Services Analytics
    - Rental Analytics
- Mobility providers to manage their vehicles
    - Vehicle Management
    - Rental Analytics

SUMMS encourages sustainability by gamifying the user's experience:

- It recommends low emission vehicles (EVs, bikes, scooters)
- It assesses the user's reliability score when returning vehicles successfully
- It rewards reliable users with discounts

## 1.2 Architectural style selected

MVC (Model View Controller) architecture is used for this system. The View handles the presentation layer, the Controller handles the input and coordination, and the Model handles the data and business logic.

The user interacts with the view, the view communicates with the controller, and the controller asks the model the data it needs. The model reads or writes to the database.

MVC was selected because it establishes a clear separation of concerns between data, presentation and logic. This also improves the software's maintainability and testability because each component can be updated or tested individually. Lastly, it allows for scalability since it's modular and can easily scale with user count.

## 1.3 Core components and responsibilities

### 1.3.1 Components

The components communicate to the Controller and then to the View through APIs.

The components of SUMMS are:

- User Access Service
- Rental System
- Vehicle Management
- External Services
    - Parking Management
    - Public Transportation Service
- Analytics System
    - Rental Analytics
    - Gateway Analytics
    - Sustainability Analytics
- Sustainability Module/Reliability Scoring System/Gamification
    - Reward successful Rental Returns
    - Recommend low CO2 emissions

### 1.3.2 Responsibilities

| Components | GRASP Pattern | Responsibilities |
|---|---|---|
| User Access Service | Information Expert<br>Controller | Manage signup, login, and subscriptions |
| Rental System | Creator<br>Information Expert<br>Controller | Rental Lifecycle (Search, Reserve + Mock Payment, Unlock, Return, History) |
| Vehicle Management | Creator<br>Information Expert<br>Controller | Add/Remove/Update Vehicle<br>Vehicle Status |
| Parking Service | Information Expert<br>Controller | Access Parking APIs (availability) |
| Public Transit | Information Expert | Access Public Transit APIs (routes/schedules) |

| | | |
|---|---|---|
| | Controller | |
| Analytics | Information Expert<br>Pure Fabrication | Rental Analytics<br>External Services Analytics (Parking/Transit) |
| Sustainability Module | Information Expert<br>Pure Fabrication | Emissions score (per vehicle trip),<br>Reliability score (successful returns),<br>Personal Statistics,<br>Rewards/Discounts |

In phase 3, the features that will be implemented are the following:

1. Complete User(s) Registration and Access
    - User registration (profile creation)
    - Authentication/login
2. Complete Rental Lifecycle
    - Vehicle search
    - Reservation
    - Payment processing (simulation is acceptable)
    - Vehicle return
3. Complete Mobility Provider Features
    - Vehicle management (add/update/remove vehicles)
    - Any provider-level operations defined in our use cases
4. Navigation to External Services
    - Functional navigation/access to:
        - Parking feature
        - Public transportation feature
          (These can remain abstracted services, but navigation must work.)
5. Partial Implementation of Analytics
    - At least one working rental-related analytic
    - At least one gateway/service-level analytic
6. Partial Implementation of Gamification/Reliability Score System
    - Carbon Footprint per Rental
    - Vehicle Return Points
    - Rewards

Important Notes

Your implementation must reflect your selected architecture(in terms of separation of concerns and components services and responsibilities).

Your team's additional custom features do not need to be fully completed in Sprint 3; however, partial implementation is encouraged.

Full implementation of additional features will be required in Sprint 4.

This sprint focuses on demonstrating that your system is operational and aligned with your design decisions.

# 2. Design Patterns

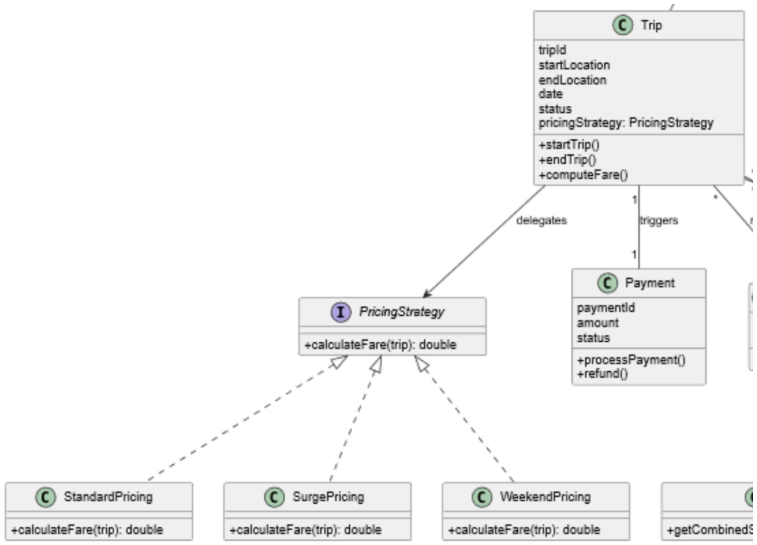
For at least four design situations:

- Description of the design problem.
- Justification of selected GOF GangOfFour pattern.
- Updated UML diagram(s).
- Consequences if the pattern were not applied.
- Reflection on flexibility, scalability, maintainability.

### *2.1 Design Pattern #1*

**Strategy Pattern: Pricing and Fare Calculation**

The original model implicitly assumed that the fare computation was part of the Trip or Payment logic, which lead to a very coupled implementation with many conditions when pricing rules vary, either by, for example, vehicle type, provider, time, or demand. By introducing the Strategy pattern, the pricing behavior is now encapsulated into an interchangeable PricingStrategy implementation such as through StandardPricing, WeekendPricing, or SurgePricing. The Trip then passes on the calculation for fares to the selected strategy at runtime. This localizes the changes, which allows to then add new pricing rules without changing the main system logic. It also makes testing easier because each pricing method can be tested on its own.




Context: Trip

Strategy: PricingStrategy

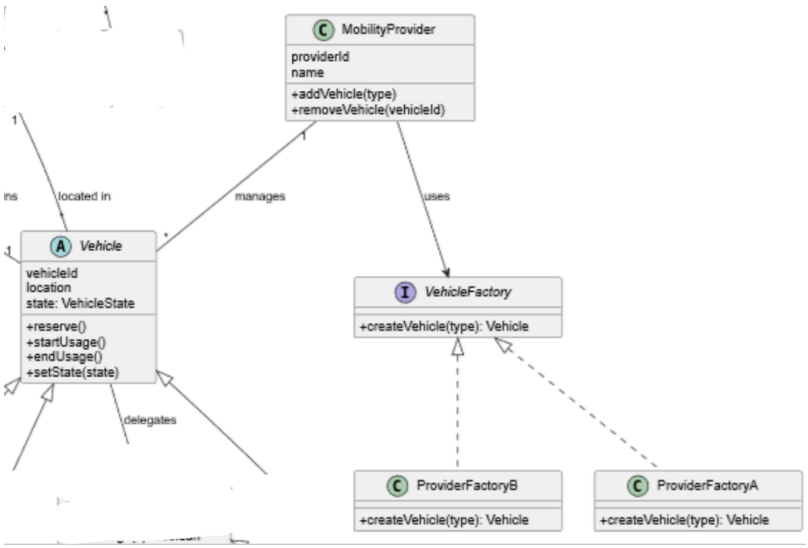
Concrete pricing classes

### *2.2 Design Pattern #2*

**Factory Pattern: Vehicle Creation by Providers**

In the base model, MobilityProvider manages vehicles but implicitly handles their creation as well. This could lead to a scattered logic for instantiating objects as well as specific provider configuration that could undesirably leak in design into other parts of the system. By introducing

a Factory, each provider then delegates the vehicle creation to a VehicleFactory. This encapsulates the construction logic, including initialization and provider-specific configs. It makes sure that adding new vehicle types or new providers does not require modifying existing code, thereby improving scalability and maintainability.



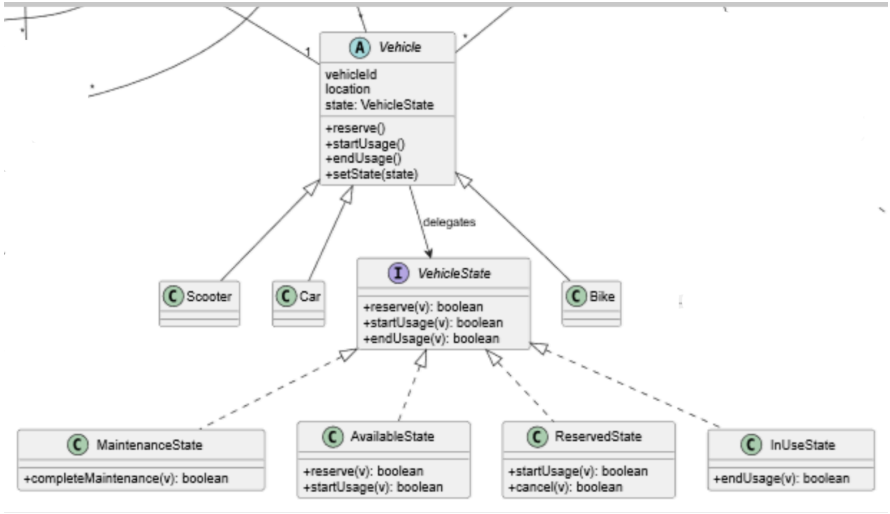

Creator: VehicleFactory

Concrete creators: ProviderFactoryA/B

### *2.3 Design Pattern #3*

**State Pattern: Vehicle Lifecycle Management**

The Vehicle status attribute suggests multiple lifecycle states, like available, reserved, in use, maintenance, etc. but without a proper structure, this would most likely result in many large conditional blocks throughout many method calls. The State pattern would replace this with a more composition-oriented approach where each state (AvailableState, ReservedState, etc.) encapsulates its own behavior. The Vehicle then delegates actions like reserve() or startUsage() to the object that describes its current state. This leads to a cleaner, more modular code and ensures that state transitions and constraints are consistently enforced. Adding a new state also no longer requires modifying existing logic, only adding a new class.




Context: Vehicle

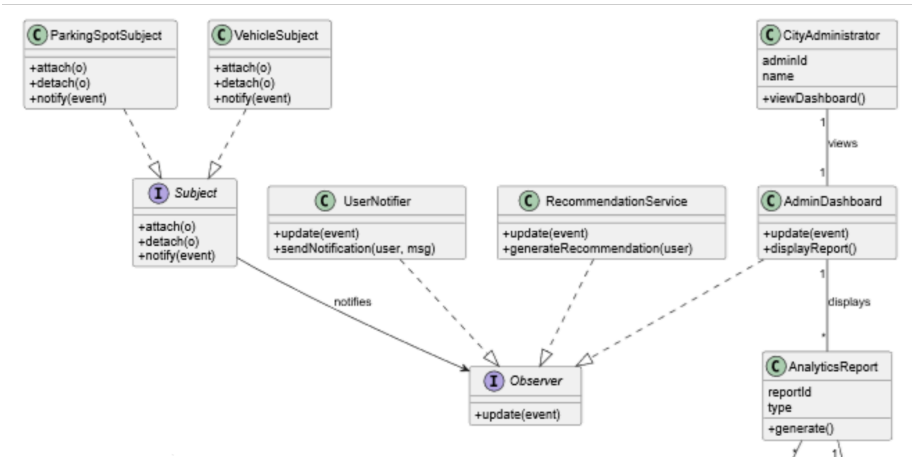
State: VehicleState

Concrete states: others

### *2.4 Design Pattern #4*

**Observer Pattern: Notifications and System Reactions**

The system requires multiple components, such as users, dashboards, recommendation services, to react to dynamic changes like vehicle availability or parking updates. Without a pattern, these interactions would create a lot of coupling between the producers of events and the receivers of events. The Observer pattern introduces Subject and Observer abstractions, allowing entities like Vehicle and ParkingSpot, via subjects, to notify multiple observers (UserNotifier, AdminDashboard, RecommendationService) without knowing their implementations. This decouples components, supports extensibility, and aligns well with event-driven architectures.



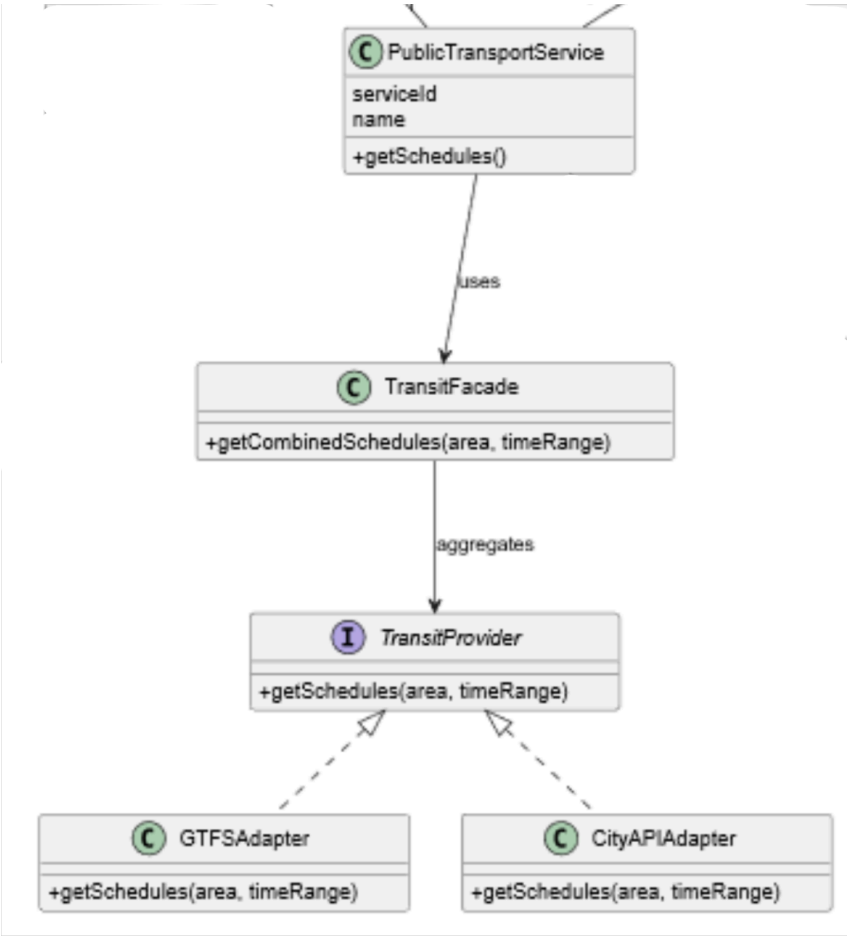

Subject: VehicleSubject

Observers: notifiers/services

### *2.5 Design Pattern #5*

**Adapter Pattern: Public Transport Integration**

Instead of having PublicTransportService directly interact with multiple public transport services, each with its own format and API, we introduce an intermediate layer that standardizes communication. Direct integration would spread API-specific logic throughout the code. By introducing an Adapter pattern, each external system is wrapped in a TransitProvider implementation (e.g., GTFSAdapter, CityAPIAdapter) that conforms to a unified interface. This approach keeps the core system clean and consistent, reduces the impact of changes in external services, and makes it easy to add or replace transit providers without affecting the rest of the application.




Target: TransitProvider
Adapters: GTFSAdapter, CityAPIAdapter

# 3 Updated Class Diagram

- Complete class diagram including attributes and methods.
- Clear integration of design patterns.
- Explanation of improvements over previous version.

**What Changed Structurally**

First, the Trip class no longer contains embedded pricing logic; instead, it maintains a reference to a PricingStrategy, to which it delegates fare computation. This replaces rigid logic with polymorphic behavior.

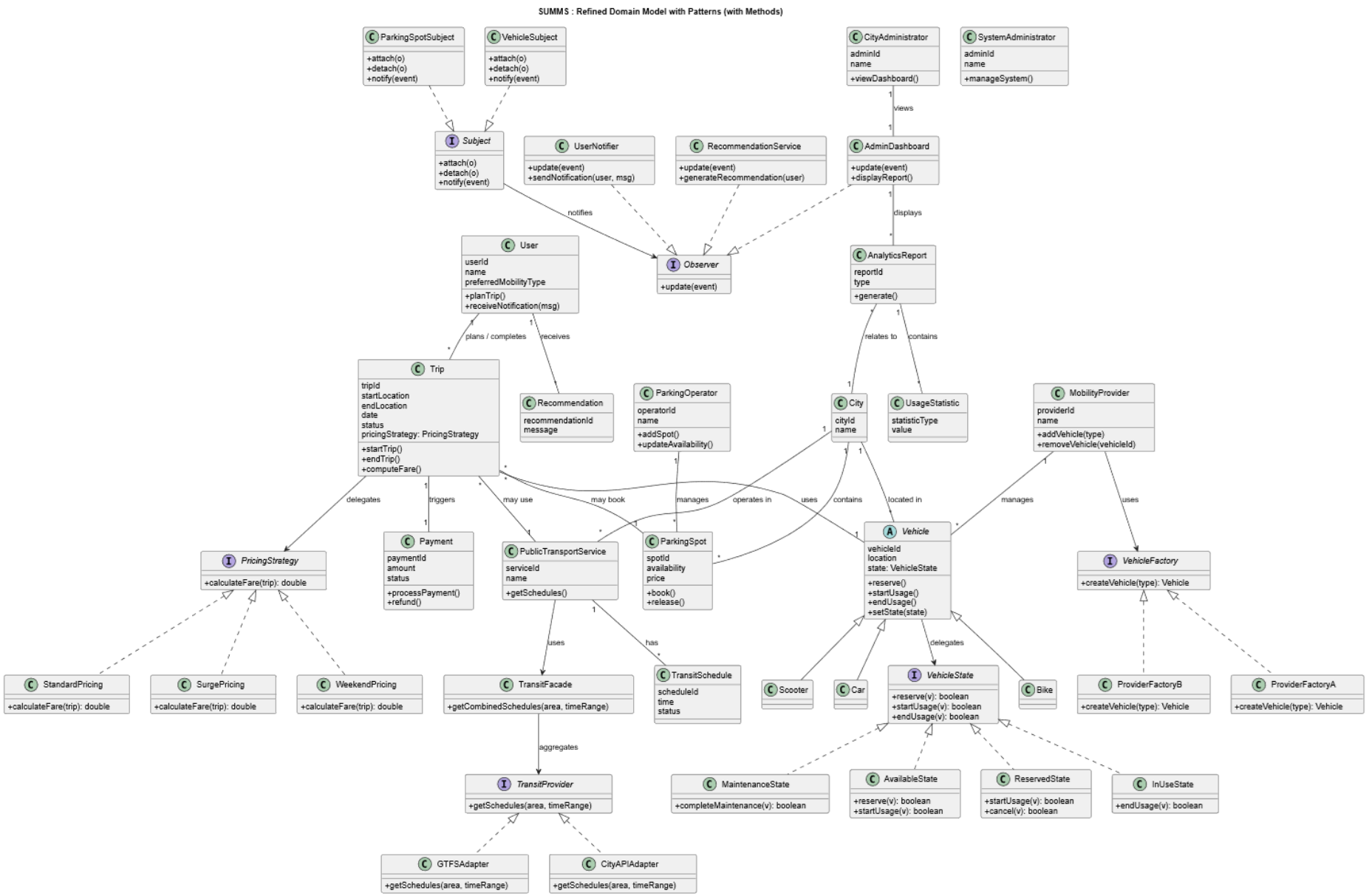
Second, MobilityProvider is no longer responsible for directly instantiating Vehicle objects. Instead, it uses a VehicleFactory, which centralizes and abstracts the creation process. This removes construction dependencies from the domain layer.

Third, the Vehicle class has been restructured to replace a simple status attribute with a composition relationship to VehicleState. All state-dependent behaviors are now delegated to concrete state classes, eliminating conditional branching and improving cohesion.

Fourth, an event-driven structure has been introduced via the Subject and Observer interfaces. Vehicle and ParkingSpot now act as event sources (through subject wrappers), notifying multiple observers such as notification services and dashboards. This removes direct dependencies between components that produce and consume system events.

Finally, the interaction with public transport systems has been refactored. Instead of directly accessing external services, PublicTransportService now communicates through a TransitFacade, which aggregates multiple TransitProvider adapters. This creates a clean boundary between the internal domain model and external systems.

Together, these changes transform the system from a tightly coupled, condition-heavy design into a modular, extensible architecture that follow core object-oriented design principles more closely such as encapsulation and separation of concerns.


SUMMS : Refined Domain Model with Patterns (with Methods)
ParkingSpotSubject
+attach(o)
+detach(o)
+notify(event)
VehicleSubject
+attach(o)
+detach(o)
+notify(event)
CityAdministrator
adminId
name
+viewDashboard()
SystemAdministrator
adminId
name
+manageSystem()
views
Subject
+attach(o)
+detach(o)
+notify(event)
UserNotifier
+update(event)
+sendNotification(user, msg)
RecommendationService
+update(event)
+generateRecommendation(user)
AdminDashboard
+update(event)
+displayReport()
notifies
displays
User
userId
name
preferredMobilityType
+planTrip()
+receiveNotification(msg)
Observer
+update(event)
AnalyticsReport
reportId
type
+generate()
plans / completes
receives
relates to
contains
Trip
tripId
startLocation
endLocation
date
status
pricingStrategy: PricingStrategy
+startTrip()
+endTrip()
+computeFare()
Recommendation
recommendationId
message
ParkingOperator
operatorId
name
+addSpot()
+updateAvailability()
City
cityId
name
UsageStatistic
statisticType
value
MobilityProvider
providerId
name
+addVehicle(type)
+removeVehicle(vehicleId)
delegates
triggers
may use
may book
manages
operates in
uses
contains
located in
manages
uses
PricingStrategy
+calculateFare(trip): double
Payment
paymentId
amount
status
+processPayment()
+refund()
PublicTransportService
serviceId
name
+getSchedules()
ParkingSpot
spotId
availability
price
+book()
+release()
Vehicle
vehicleId
location
state: VehicleState
+reserve()
+startUsage()
+endUsage()
+setState(state)
VehicleFactory
+createVehicle(type): Vehicle
uses
has
delegates
StandardPricing
+calculateFare(trip): double
SurgePricing
+calculateFare(trip): double
WeekendPricing
+calculateFare(trip): double
TransitFacade
+getCombinedSchedules(area, timeRange)
TransitSchedule
scheduleId
time
status
Scooter
Car
VehicleState
+reserve(v): boolean
+startUsage(v): boolean
+endUsage(v): boolean
Bike
ProviderFactoryB
+createVehicle(type): Vehicle
ProviderFactoryA
+createVehicle(type): Vehicle
aggregates
TransitProvider
+getSchedules(area, timeRange)
MaintenanceState
+completeMaintenance(v): boolean
AvailableState
+reserve(v): boolean
+startUsage(v): boolean
ReservedState
+startUsage(v): boolean
+cancel(v): boolean
InUseState
+endUsage(v): boolean
GTFSAdapter
+getSchedules(area, timeRange)
CityAPIAdapter
+getSchedules(area, timeRange)